Московский государственный университет имени М. В. Ломоносова

Факультет вычислительной математики и кибернетики

Решение задачи Дирихле методом фиктивных областей

Отчёт по курсу
«Суперкомпьютерное моделирование и технологии»

Вариант 5
Последовательная реализация и OpenMP
Пункты 1–3 практикума

Выполнил: Орлов Сергей
Группа 681

Москва 2026

1 Постановка задачи и метод фиктивных областей

Цель работы — решить задачу Дирихле для уравнения Пуассона в четырёхугольной области методом фиктивных областей и исследовать эффективность OpenMP-реализации. В работе приведены разностная схема, описание метода решения и результаты вычислений на IBM Polus. Максимальное ускорение на сетке 800 × 1200 составило 5,608 при использовании 32 потоков.

Согласно варианту 5 задания [1], область D ограничена четырёхугольником с вершинами A(0,0), B(3,0), C(3,2), D(1,3). Требуется найти решение задачи

$$-\Delta u = 1 \text{ в } D, \quad u = sin(x + y) \text{ на } \gamma = \partial D.$$

Область задаётся неравенствами $y \geq 0$, $x \leq 3$, $x + 2y \leq 7$, $y \leq 3x$. Её площадь по формуле многоугольника равна 6,5. Для расчёта выбран прямоугольник $\Pi = (-1,4) \times (-1,4)$, строго содержащий замыкание D. Внешняя граница прямоугольника обозначается Г.

Метод фиктивных областей с продолжением по младшему коэффициенту заменяет исходную задачу задачей в прямоугольнике. В качестве продолжения граничной функции используется $v_0(x,y) = \sin(x+y)$:

$$-\Delta v + Q(x,y)(v - v_0) = F(x,y) \text{ в } \Pi \setminus \gamma, \quad v = v_0 \text{ на } \Gamma.$$

$$Q = 0 \text{ в } D, \quad Q = \varepsilon^{-2} \text{ в } \Pi \setminus D;$$

$$F = 1 \text{ в } D, \quad F = 0 \text{ в } \Pi \setminus D.$$

На внутренней границе γ требуется непрерывность v и нормальной компоненты потока −grad(v). Эти условия связывают решения в исходной и фиктивной областях; отдельное принудительное условие Дирихле на γ в расширенной задаче не задаётся.

Внутри исходной области получается уравнение Пуассона; в фиктивной области большой потенциал приближает v к продолжению v_0. Согласно постановке [1], погрешность метода фиктивных областей в D имеет

порядок O(ε). Эта оценка сама по себе не является оценкой полной ошибки разностного решения.

Функция sin(x+y) задаёт граничные значения, но не является точным решением данной задачи: $-\Delta\sin(x+y) = 2\sin(x+y)$, что не равно постоянной правой части 1. Поэтому сравнение с ней во всей области нельзя использовать как проверку ошибки решения.

2 Разностная схема

На замыкании П вводится равномерная сетка. Числа M и N обозначают количество интервалов по координатам; полная сетка содержит (M+1)(N+1) узлов, а система — (M−1)(N−1) неизвестных:

$$x_i = -1 + ih_1, \quad i = 0, \dots, M; \quad y_j = -1 + jh_2, \quad j = 0, \dots, N;$$

$$h_1 = 5/M, \quad h_2 = 5/N, \quad h = max(h_1, h_2), \quad \varepsilon = h^2.$$

Для внутреннего узла рассматривается прямоугольная ячейка Π_{ij} с центром (x_i, y_j) и размерами $h_1 \times h_2$. Площадь её пересечения с D обозначается S_{ij}. Доля исходной области в ячейке равна $\theta_{ij} = S_{ij}/(h_1 h_2)$. Коэффициенты схемы получаются усреднением по ячейке:

$$F_{ij} = \theta_{ij}, \quad Q_{ij} = (1 - \theta_{ij})/\varepsilon^2.$$

При f = 1 вычисление F_{ij} через площадь точно соответствует интегралу кусочно-постоянной правой части. Площадь находится отсечением выпуклого четырёхугольника четырьмя полуплоскостями ячейки. Площадь результирующего многоугольника вычисляется относительно его первой вершины, что уменьшает потерю точности для мелких ячеек.

Аппроксимация оператора Лапласа выполняется пятиточечным шаблоном:

$$(2/{h_1}^2 + 2/{h_2}^2 + Q_{ij})w_{ij}$$
$$- (w_{i-1,j} + w_{i+1,j})/{h_1}^2 - (w_{i,j-1} + w_{i,j+1})/{h_2}^2$$
$$= F_{ij} + Q_{ij} v_0(x_i, y_j).$$

На Г задаются значения $w = v_0$. Известные граничные компоненты переносятся в правую часть с положительными коэффициентами h_1^{-2} или h_2^{-2}. В системе остаются только внутренние неизвестные; при применении A к внутреннему вектору внешние компоненты считаются нулевыми.

Полученная система $Aw = B$ имеет симметричную положительно определённую матрицу. Матрица целиком не хранится: произведение Aw вычисляется непосредственно по шаблону. Это обеспечивает линейный по числу неизвестных объём памяти.

3 Метод сопряжённых градиентов

Для решения $Aw = B$ используется метод сопряжённых градиентов с диагональным предобуславливанием. Чтобы не смешивать предобуславливатель с обозначением геометрической области D, обозначим его P:

$$P_{ij,\,ij} = 2/h_1^{\,2} + 2/h_2^{\,2} + Q_{ij}.$$

Начальное приближение $w^{(0)}$ равно нулю. Начальная невязка $r^{(0)} = B$, предобусловленная невязка $z^{(0)} = P^{-1}r^{(0)}$, направление $p^{(0)} = z^{(0)}$. Далее выполняются итерации:

$$\alpha_k = (r^{(k)}, z^{(k)}) \,/\, (Ap^{(k)}, p^{(k)}),$$

$$w^{(k+1)} = w^{(k)} + \alpha_k p^{(k)}, \qquad r^{(k+1)} = r^{(k)} - \alpha_k A p^{(k)},$$

$$z^{(k+1)} = P^{-1} r^{(k+1)},$$

$$\beta_k = (r^{(k+1)}, z^{(k+1)}) \,/\, (r^{(k)}, z^{(k)}),$$

$$p^{(k+1)} = z^{(k+1)} + \beta_k p^{(k)}.$$

Скалярное произведение и норма определяются на внутренних узлах с весом $h_1 h_2$. Постоянный вес сокращается в формулах α и β, поэтому в этих отношениях можно использовать обычные суммы произведений компонент.

$$(a, b) = h_1 h_2 \sum_i \sum_j a_{ij} b_{ij}, \qquad \|a\|^{E} = \sqrt{(a, a)}.$$

Итерационный процесс останавливается по условию задания, с одинаковым допуском $\delta = 10^{-8}$ во всех опытах:

$$\|w^{(k+1)} - w^{(k)}\|^E = |\alpha_k| \sqrt{(h_1 h_2 \sum_i \sum_j (p_{ij}^{(k)})^2)} < \delta.$$

Суммарное число попыток итераций не превышает числа неизвестных. Дополнительно проверяются конечность коэффициентов и положительность (Ap,p). Норма шага не совпадает с нормой невязки и не даёт прямой гарантии ошибки решения менее δ.

Для контроля численной устойчивости отслеживается функционал J(w) = 0,5(Aw,w) − (B,w) = −0,5(B+r,w). При росте сверх допуска округления шаг отклоняется, восстанавливается предыдущее приближение, явно пересчитывается невязка и CG перезапускается. Допуск зависит от числа слагаемых и суммы их модулей. Защитный предел составляет 32 перезапуска; его достижение означает ошибку завершения. В итоговых 80 запусках перезапусков не было.

4 Реализация и измерение времени

Разработаны две программы на C: solver_seq.c и solver_omp.c. Неизвестные хранятся в одномерном массиве по индексу (i−1)(N−1)+(j−1). Геометрия, коэффициенты системы, начальное приближение, предобуславливатель и критерий остановки в обеих версиях одинаковы.

В OpenMP-версии распараллелены построение системы и применение оператора по внешнему индексу i, а также векторные операции по линейному индексу. Для сумм площади, скалярных произведений и функционала используется reduction. Применяется статическое распределение итераций schedule(static). Каждый элемент выходного массива записывается одной нитью; локальные массивы отсечения создаются отдельно для каждого вызова.

На весь итерационный решатель создаётся одна параллельная область. Циклы внутри неё распределяются директивой omp for; зависимые операции разделяются неявными барьерами. Общие скаляры обновляются в single с

барьером. Независимые суммирования объединены в общие проходы по массивам. Все нити согласованно выполняют проверки остановки и ветви перезапуска.

Измеряется только вызов conjugate_gradient: инициализация рабочих векторов, итерации CG, контроль функционала, синхронизация и вычисление итоговой рекуррентной нормы невязки внутри решателя. Построение системы и однократное выделение памяти выполняются до таймера. Освобождение памяти, явная итоговая диагностика и запись CSV выполняются после него. Граница измерения одинакова для обеих программ.

Последовательный таймер использует CLOCK_MONOTONIC в Linux, таймер OpenMP — omp_get_wtime. Для таблиц используется внутреннее поле Algorithm time. Служебные CPU time и Run time из отчёта LSF характеризуют другую границу измерения и в таблицу не подставляются.

На IBM Polus использовался компилятор GCC 4.8.5. Обе программы собирались с уровнем оптимизации -O3, для OpenMP-версии дополнительно использовался флаг -fopenmp. Команды компиляции:

```
gcc -O3 -std=c11 -Wall -Wextra -Wpedantic \
    solver_seq.c -o solver_seq -lm
gcc -O3 -std=c11 -Wall -Wextra -Wpedantic -fopenmp \
    solver_omp.c -o solver_omp -lm
```

5 Организация эксперимента и малые сетки

Расчёты проведены 30 октября 2026 года на IBM Polus через планировщик LSF. По сохранённому lscpu узлы имеют архитектуру ppc64le, процессоры POWER8NVL, два сокета по 10 физических ядер и восемь аппаратных потоков на ядро. Всего ОС видит 160 логических процессоров. Эти характеристики описывают узел, а не число ресурсов, выделенных конкретному заданию.

Все конфигурации выполнялись последовательно в одном задании LSF на узле polus-c3. Для задания выделены пять слотов с размещением на одном узле. Число потоков OpenMP не следует отождествлять с числом выделенных физических ядер.

Использовались настройки OMP_DYNAMIC=FALSE и OMP_PROC_BIND=TRUE; OMP_PLACES не задавалась. Число потоков устанавливалось через OMP_NUM_THREADS и аргумент программы. Для каждой конфигурации выполнено пять запусков. В таблицах приведено минимальное время работы итерационного решателя из пяти повторов.

Таблица 1 — Проверка на малых сетках и минимальные времена Polus

Версия	Сетка M × N	Нити	Итерации	Время с
seq	20 × 20	1	36	0,000310
seq	40 × 40	1	72	0,002588
seq	80 × 80	1	140	0,020398
omp	80 × 80	1	140	0,021746
omp	80 × 80	4	140	0,016917
omp	80 × 80	16	140	0,014267

В последовательной программе получены 36, 72 и 140 итераций при сгущении сетки 20 × 20 → 40 × 40 → 80 × 80. На 80 × 80 OpenMP-версия во всех случаях также выполнила 140 итераций. При одной нити значения последовательного и параллельного решений совпали в сохранённых CSV; при 4 и 16 нитях максимальные расхождения составили $2{,}00 \cdot 10^{-15}$ и $1{,}67 \cdot 10^{-15}$.

На сетке 80 × 80 минимум последовательного времени равен 0,020398 с. OpenMP с одной нитью медленнее: 0,021746 с; с четырьмя и шестнадцатью нитями получены 0,016917 и 0,014267 с, то есть ускорение около 1,21 и 1,43. Накладные расходы OpenMP остаются заметными на малом объёме работы.

6 Результаты вычислительных экспериментов

Вычислительные эксперименты проведены на сетках 400 × 600 и 800 × 1200 для всех значений числа потоков, указанных в задании. На каждой сетке

также измерено время последовательной программы. Ускорение вычислялось по формуле

$$S_{\text{p}} = Tseq, min \,/\, TOMP, p, min.$$

Здесь Tseq,min и TOMP,p,min — минимальные времена из пяти повторов соответствующей конфигурации.

Последовательные минимумы: Tseq = 4,992032 с для 400 × 600 и Tseq = 41,579893 с для 800 × 1200. Все конфигурации имеют одинаковый допуск $\delta = 10^{-8}$ и одинаковую границу таймера. Ускорение относительно однопоточной OpenMP-версии здесь не используется.

Таблица 2 — Результаты OpenMP-программы на IBM Polus

Нити	Сетка M × N	Итерации	Время с	Ускорение
2	400 × 600	815	4,254257	1,1734
4	400 × 600	815	4,013257	1,2439
8	400 × 600	815	3,658433	1,3645
16	400 × 600	815	1,846089	2,7041
4	800 × 1200	1598	31,976119	1,3003
8	800 × 1200	1598	29,117560	1,4280
16	800 × 1200	1598	14,653687	2,8375
32	800 × 1200	1598	7,414812	5,6077

На сетке 400 × 600 все расчёты завершились за 815 итераций, на сетке 800 × 1200 — за 1598 итераций. Число итераций не зависело от числа потоков. Во всех 80 запусках выполнен критерий остановки; перезапуски метода не потребовались. Согласованность реализаций проверена сравнением значений решений, результаты которого приведены в разделе 9.

На сетке 400 × 600 ускорение возрастает от 1,173 на двух нитях до 2,704 на шестнадцати. На 800 × 1200 получены 1,300 на четырёх нитях и 5,608 на 32 нитях. Все конфигурации основной большой таблицы быстрее

соответствующей последовательной базы. Эти показатели относятся к данной реализации и режиму выделения ресурсов.

Таким образом, эксперимент включает 16 конфигураций по пять повторов. Расчёты на малых сетках использованы для проверки реализации, а на больших — для оценки производительности.

7 Анализ ускорения и устойчивости измерений

Рисунок 1 показывает ускорение по минимуму из пяти повторов. Пунктирная линия S = 1 соответствует времени последовательной программы.

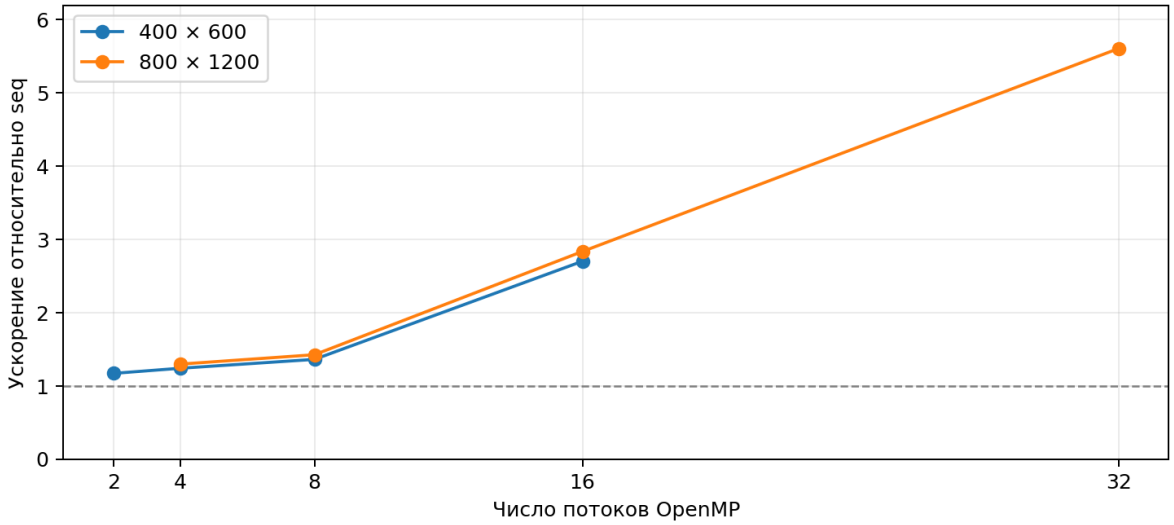


Рисунок 1 — Ускорение OpenMP относительно последовательной программы

Таблица 3 — Разброс времени на сетке 800 × 1200

Нити	Минимум с	Медиана с	Максимум с
4	31,976	31,993	32,017
8	29,118	29,122	29,130
16	14,654	14,688	14,696
32	7,415	7,424	7,434

На сетке 800 × 1200 при 32 потоках минимальное время составило 7,415 с, медиана — 7,424 с, максимальное время — 7,434 с. Размах равен примерно 0,25% минимального времени. Для последовательной программы

время изменялось от 41,580 до 41,869 с. В пределах выполненной серии измерения достаточно устойчивы.

Полученные ускорения характеризуют данную реализацию и условия выполнения на IBM Polus. При их интерпретации следует учитывать накладные расходы синхронизации, использование аппаратных потоков SMT и возможное влияние нагрузки других заданий. Нагрузка соседних заданий отдельно не измерялась.

8 Приближённое решение

На рисунке 2 приведено решение последовательной программы на самой подробной сетке 800 × 1200. Показаны только узлы исходной области D; фиктивная область исключена из изображения. Чёрная линия обозначает границу четырёхугольника.

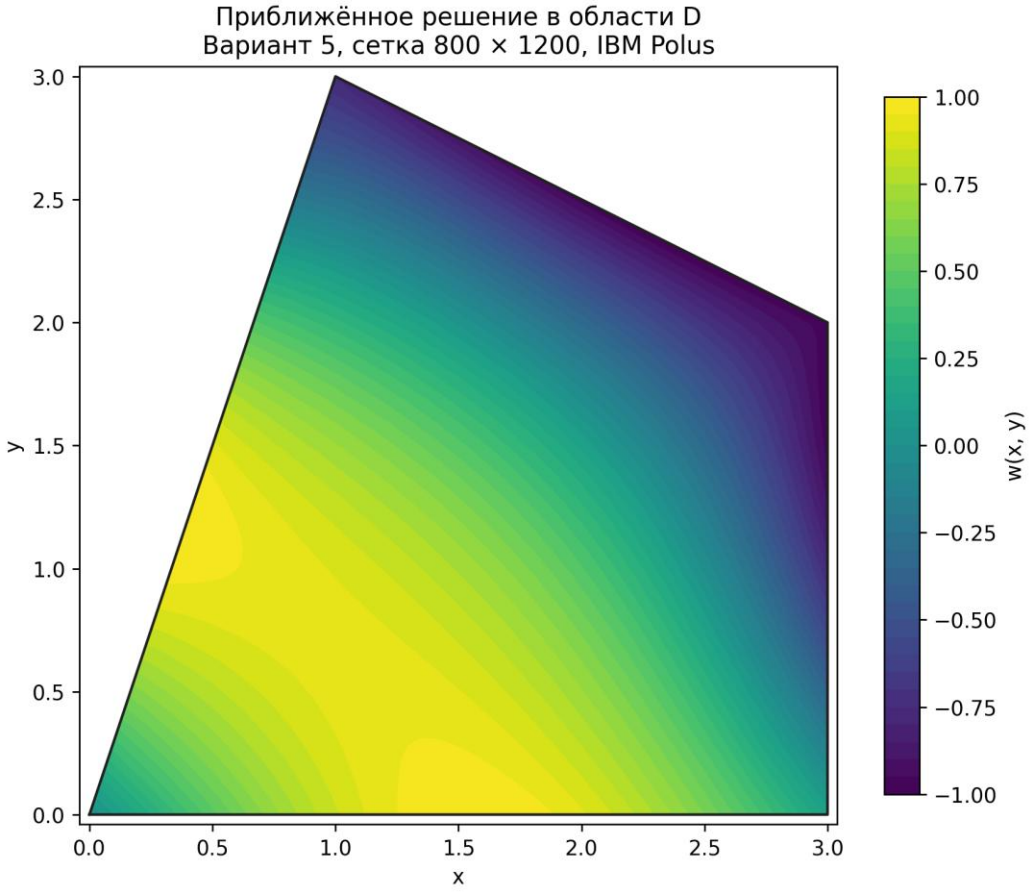


Рисунок 2 — Приближённое решение варианта 5 внутри области D

Для визуализации использовано решение последовательной программы на сетке 800 × 1200. Полная сетка содержит 962 001 узел, из которых 250 161 принадлежит замыканию области D.

Для внутренних узлов П получены $\|w\|^E$ = 3,405936599488 и $\|w\|^c$ = 0,999999996264, совпадающие с логом запуска. В исходной области значения лежат приблизительно в диапазоне от −0,999999328 до 0,999997754. На внешней границе П отклонение сохранённых значений от sin(x+y) не превышает $1{,}12 \cdot 10^{-16}$.

Явная невязка $\|B-Aw\|^E$ на этой сетке равна $2{,}27514 \cdot 10^{-4}$, а норма последнего шага — $9{,}82483 \cdot 10^{-9}$. Это не противоречие: заданный критерий ограничивает изменение приближения, а не норму невязки. Рисунок иллюстрирует численное решение, но не заменяет оценку ошибки относительно непрерывной задачи.

9 Проверка корректности и выводы

В итоговом эксперименте проверены сходимость, нормы, восстановленная площадь и согласованность последовательной и OpenMP-версий. Максимальные расхождения значений решений по сохранённым сравнениям приведены в таблице 4.

Таблица 4 — Максимальное расхождение последовательного и OpenMP-решений

Сетка	Проверенные нити	Максимум |wseq − wOMP|
80 × 80	1, 4, 16	$2{,}00 \cdot 10^{-15}$
400 × 600	2, 4, 8, 16	$1{,}49 \cdot 10^{-13}$
800 × 1200	4, 8, 16, 32	$3{,}83 \cdot 10^{-13}$

Изменение порядка суммирования при reduction приводит к малым различиям округления, не меняющим число итераций в этих опытах. Точное непрерывное решение неизвестно; приведённые сравнения проверяют согласованность реализаций, а не абсолютную ошибку метода.

Выполнены пункты 1–3 практикума: последовательная программа проверена на сгущающихся сетках; OpenMP-программа сопоставлена с ней на 1/4/16 нитях; заполнена таблица больших расчётов IBM Polus. Максимальное ускорение составило 2,704 для 400 × 600 на 16 нитях и 5,608 для 800 × 1200 на 32 нитях. Повторы устойчивы, но интерпретация масштабирования должна учитывать SMT, привязку потоков и фактическую загрузку узла.

Пункты, связанные с MPI, двумерной декомпозицией и гибридной MPI/OpenMP-программой, в состав выполненного этапа не входят. Результаты таблиц MPI и гибридной реализации здесь не приводятся.

Использованные материалы

[1] Мокин А. Ю. Задание по курсу «Суперкомпьютерное моделирование и технологии». Сентябрь 2026. Разделы 2–6 и таблица 1 приложения.